

Proyecto HealthTech

Dispensador Inteligente de Medicamentos

Documento de Diseño y Especificación Técnica

Autores:

Stefanía García López
Juan Esteban Jiménez Daza

Universidad Nacional de Colombia

Versión:	2.0.0
Fecha:	Junio de 2026
Estado:	Documento Técnico
Clasificación:	Uso Académico
Plataforma:	Raspberry Pi Zero 2W
Arquitectura:	Embedded Linux Monolítica

Índice

1. Introducción	2
2. Descripción del Proyecto	3
2.1. Contexto e Impacto Social	3
2.2. Objetivo General	3
2.3. Objetivos Específicos	3
2.4. Arquitectura General del Sistema	3
2.5. Flujo General de Usuario	4
3. Especificación de Requerimientos	4
3.1. Requerimientos Funcionales (RF)	4
3.2. Requerimientos No Funcionales (RNF)	5
4. Arquitectura de Hardware y Software	5
4.1. Selección de Hardware	5
4.2. Justificación del Servomotor SG90	6
4.3. Celda de Carga y Módulo HX711	6
4.4. Arquitectura de Software	7
4.5. Diagrama de Bloques de Hardware	7
4.6. Arquitectura Software del Sistema	8
4.7. Interfaces Eléctricas	8
5. Diagramas de Flujo del Sistema	9
5.1. Flujo Operativo Principal	9
5.2. Flujo de Lectura y Calibración del HX711	10
5.3. Flujo de Notificación con Tolerancia a Fallos de Red	11
6. Plan de Verificación y Pruebas	11
6.1. Estrategia General de Pruebas	11
6.2. Pruebas Unitarias por Módulo	12
6.2.1. PU-01: Módulo de Control del Servomotor SG90	12
6.2.2. PU-02: Módulo HX711 / Celda de Carga	13
6.2.3. PU-04: Módulo de Planificador de Horarios	13
6.2.4. PU-06: Módulo de Registro de Eventos (Logs)	14
6.3. Pruebas de Integración	14
6.4. Criterios de Aceptación	14

1. Introducción

La adherencia terapéutica constituye uno de los principales desafíos de los sistemas modernos de salud, especialmente en adultos mayores, pacientes polimedicados y personas con enfermedades crónicas que requieren tratamientos farmacológicos continuos y periódicos. Según la Organización Mundial de la Salud (OMS), en enfermedades de larga evolución la adherencia promedio en países desarrollados es apenas del 50 %, cifra que empeora considerablemente en contextos de menor acceso a servicios sanitarios. Las consecuencias de este incumplimiento incluyen progresión de enfermedades, hospitalizaciones evitables, deterioro de la calidad de vida y costos elevados para los sistemas de salud.

Entre los factores que contribuyen a la falta de adherencia se identifican: el olvido de horarios de medicación, la confusión entre medicamentos cuando el paciente debe consumir múltiples fármacos simultáneamente (*polifarmacia*), la ausencia de supervisión continua por parte del cuidador, y la falta de mecanismos automáticos de recordatorio o verificación. Esta situación se agrava en poblaciones con deterioro cognitivo leve o moderado, quienes frecuentemente no cuentan con la capacidad de gestionar autónomamente su régimen farmacológico.

El presente proyecto propone el diseño e implementación de un **dispensador inteligente semanal de medicamentos**, construido sobre una plataforma embebida Linux basada en Raspberry Pi Zero 2W. El sistema automatiza la dispensación de medicamentos mediante un mecanismo de *carrusel rotativo* controlado por un servomotor SG90, organizado en siete compartimentos correspondientes a los días de la semana. Cada evento de dispensación es acompañado por la confirmación de retiro del medicamento se verifica mediante una **celda de carga conectada al módulo HX711**, que detecta la variación de peso en el compartimento activo al momento en que el paciente extrae su dosis.

La relevancia de este proyecto radica en varios aspectos:

- **Accesibilidad tecnológica:** Al basarse en hardware de bajo costo y software libre, la solución puede replicarse y escalarse sin dependencia de plataformas comerciales propietarias.
- **Autonomía del paciente:** El sistema permite que personas mayores o con limitaciones cognitivas leves gestionen su medicación de forma independiente, con mínima intervención del cuidador.
- **Verificación objetiva:** A diferencia de sistemas basados únicamente en alarmas, la celda de carga aporta una confirmación física y cuantitativa del retiro del medicamento, reduciendo falsos positivos.
- **Integración con Linux embebido:** El proyecto permite aplicar de forma concreta los conceptos de programación de sistemas Linux embebidos, incluyendo manejo de GPIO, comunicación I2C/SPI, tareas concurrentes, administración de servicios con `systemd` y conectividad de red.

La solución tecnológica propuesta es modular, reutilizable y de fácil mantenimiento, capaz de ejecutar múltiples tareas concurrentes: control mecánico, planificación temporal, adquisición de datos del sensor de peso y conectividad remota para mensajería.

2. Descripción del Proyecto

2.1. Contexto e Impacto Social

El incumplimiento de tratamientos médicos representa un problema recurrente dentro de la salud pública global. Diversos pacientes, particularmente adultos mayores, personas con deterioro cognitivo leve o pacientes polimedicados, presentan dificultades para recordar horarios, identificar dosis y mantener continuidad terapéutica.

Con el objetivo de mitigar esta problemática, el proyecto *HealthTech* propone el diseño de un dispensador inteligente de medicamentos de funcionamiento semanal, compuesto por siete compartimentos físicos asociados a los días de la semana.

El sistema incorpora un mecanismo de asistencia con un sistema de verificación de extracción basado en una celda de carga y el módulo de adquisición HX711, que cuantifica la masa del compartimento y detecta la variación de peso al momento del retiro del medicamento.

Desde una perspectiva social, la propuesta busca mejorar la adherencia terapéutica, aumentar la autonomía del paciente y reducir errores asociados a la administración manual de medicamentos.

2.2. Objetivo General

Diseñar e implementar un dispensador inteligente semanal de medicamentos basado en Raspberry Pi Zero 2W, capaz de automatizar la entrega de dosis programadas mediante un sistema de carrusel rotativo, detectar la extracción del medicamento a través de una celda de carga y el módulo HX711.

2.3. Objetivos Específicos

1. Diseñar un sistema mecánico rotativo compuesto por siete compartimentos correspondientes a cada día de la semana, accionado por el servomotor SG90.
2. Implementar un sistema de control embebido basado en Raspberry Pi Zero 2W utilizando Linux y Python.
3. Implementar una celda de carga conectada al módulo HX711 para la confirmación de retiro del medicamento mediante detección de variación de peso.
4. Registrar eventos de dispensación en archivos locales con marcas temporales.
5. Diseñar un sistema configurable de horarios de administración sin reinicio del servicio.

2.4. Arquitectura General del Sistema

El sistema adopta una arquitectura centralizada monolítica basada en Raspberry Pi Zero 2W, eliminando la necesidad de un microcontrolador externo.

La Raspberry Pi Zero 2W ejecuta un sistema operativo Linux (*Raspberry Pi OS*), encargándose simultáneamente de:

- Gestión de horarios y eventos de dispensación.
- Control del servomotor SG90 encargado del movimiento del carrusel semanal.

- Lectura y procesamiento de señales del módulo HX711 conectado a la celda de carga.
- Registro de eventos en archivos locales.
- Configuración externa desde CLI o interfaz web ligera.

La dispensación se realiza mediante un sistema mecánico de carrusel compuesto por siete compartimentos físicos, uno correspondiente a cada día de la semana. El servomotor posiciona el compartimento correspondiente frente al punto de acceso del usuario según el horario programado.

El desplazamiento angular del carrusel se define mediante:

$$\theta = \frac{360^\circ}{7}$$

lo que corresponde a un desplazamiento angular aproximado de:

$$\theta \approx 51,43^\circ$$

para cada compartimento semanal.

2.5. Flujo General de Usuario

El flujo operacional del sistema se resume de la siguiente manera:

1. El cuidador configura horarios y parámetros de medicación mediante una interfaz local o remota.
2. La Raspberry Pi monitorea continuamente el reloj del sistema.
3. Cuando se alcanza la hora programada, se reproduce una alerta de texto: *“Es hora de tomar su medicamento”*.
4. El servomotor SG90 rota el carrusel hacia el compartimento correspondiente al día actual.
5. La celda de carga registra el peso inicial del compartimento (con medicamento).
6. El sistema espera una disminución de peso que indique el retiro del medicamento.
7. El evento se registra localmente con la lectura de peso antes y después.

3. Especificación de Requerimientos

3.1. Requerimientos Funcionales (RF)

- **RF-1:** El sistema debe emitir una alerta de mensaje al menos 60 segundos antes de la dispensación programada.
- **RF-2:** El sistema debe posicionar automáticamente el carrusel rotativo en el compartimento correspondiente al día actual, mediante el servomotor SG90.
- **RF-3:** El sistema debe detectar la extracción del medicamento mediante la variación de peso registrada por la celda de carga y el módulo HX711.

- **RF-5:** El sistema debe permitir modificación dinámica de horarios sin reiniciar el servicio.
- **RF-6:** El sistema debe registrar todos los eventos con marca temporal UTC, incluyendo la lectura de peso antes y después del retiro.
- **RF-7:** El sistema debe operar aun cuando exista pérdida temporal de conexión a Internet, almacenando notificaciones pendientes.
- **RF-8:** El sistema debe calibrar la celda de carga al inicio de cada jornada para asegurar lecturas precisas de peso.

3.2. Requerimientos No Funcionales (RNF)

- **RNF-1:** El sistema deberá mantener una disponibilidad mínima del 99.5 %.
- **RNF-2:** El tiempo entre activación del evento y movimiento del servo no deberá superar 200 ms.
- **RNF-3:** El software deberá ejecutarse sobre Raspberry Pi OS con Python 3.9 o superior.
- **RNF-4:** El sistema deberá consumir menos de 5W en estado de espera.
- **RNF-5:** El sistema deberá tolerar fallos temporales de red sin pérdida de eventos.
- **RNF-6:** El sistema deberá reiniciar automáticamente servicios críticos ante fallos, gestionado mediante `systemd`.
- **RNF-7:** La resolución mínima de la celda de carga deberá ser de 1 gramo, suficiente para distinguir la presencia o ausencia de pastillas estándar.

4. Arquitectura de Hardware y Software

4.1. Selección de Hardware

La implementación del sistema se basa en una arquitectura embebida centralizada cuyo núcleo computacional corresponde a una Raspberry Pi Zero 2W. Esta plataforma ejecuta simultáneamente tareas de control lógico, planificación temporal, conectividad de red, procesamiento de eventos y control electromecánico del sistema de dispensación.

La Tabla 1 resume los componentes principales empleados.

Tabla 1: Selección de hardware del sistema

Componente	Modelo	Función
Unidad principal	Raspberry Pi Zero 2W	Control central del sistema, lógica de dispensación y gestión de eventos
Servomotor	SG90	Movimiento angular del carrusel semanal de siete compartimentos
Sensor de extracción	Celda de carga + HX711	Detección del retiro del medicamento por variación de peso
Almacenamiento	MicroSD 16 GB o superior	Sistema operativo y persistencia de registros
Conectividad	Wi-Fi integrado	
Alimentación	Fuente 5V regulada (2A mínimo)	Suministro energético del sistema
Pulsador	Push button GPIO	Acción manual de confirmación o mantenimiento

4.2. Justificación del Servomotor SG90

El mecanismo de dispensación utiliza un servomotor SG90 para el posicionamiento angular del carrusel semanal. Este servo de bajo perfil presenta un torque de salida de 1.8 kgf·cm suficiente para rotar la plataforma de siete compartimentos considerando que las masas involucradas (medicamentos y estructura ligera en PLA o cartón) son reducidas.

El movimiento del servo se realiza mediante modulación por ancho de pulso (PWM), generada directamente desde un pin GPIO de la Raspberry Pi. El desplazamiento angular entre compartimentos está dado por:

$$\theta = \frac{360^\circ}{7} \approx 51,43^\circ$$

Dado que el SG90 opera en un rango de 0° a 180° , la posición de cada día d (con $d \in \{0, 1, \dots, 6\}$) se calcula acumulando rotaciones:

$$\alpha_d = d \times 51,43^\circ$$

Para un carrusel de 360° , se recomienda implementar la rotación de forma incremental, moviendo el servo de su posición actual a la siguiente en pasos para evitar saltos bruscos que puedan desplazar el contenido de los compartimentos.

4.3. Celda de Carga y Módulo HX711

La confirmación del retiro del medicamento se realiza mediante una **celda de carga** de bajo perfil ubicada bajo el punto de acceso del carrusel (o bajo cada compartimento, según el diseño mecánico). El módulo **HX711** actúa como amplificador y convertidor analógico-digital (ADC) de 24 bits, comunicándose con la Raspberry Pi mediante dos pines GPIO (DATA y CLK) en un protocolo serial propietario.

El proceso de verificación sigue la siguiente lógica:

1. Al posicionarse el compartimento del día, se adquiere una lectura de **peso de referencia** W_0 (compartimento con medicamento).
2. El sistema aguarda durante la ventana de tiempo configurada.
3. Al final de la ventana, se adquiere una lectura de **peso final** W_f .
4. Si la diferencia $\Delta W = W_0 - W_f$ supera un umbral mínimo $\Delta W_{\min}$ (configurable, por defecto 1 g), se considera que el medicamento fue retirado exitosamente.

La calibración del HX711 se realiza al inicio mediante un *factor de escala* determinado con una masa de referencia conocida.

4.4. Arquitectura de Software

La arquitectura software se implementa sobre Linux embebido utilizando Raspberry Pi OS. El sistema se desarrolla principalmente en Python, aprovechando su amplia compatibilidad con bibliotecas de automatización, GPIO, síntesis de voz y conectividad remota.

Las responsabilidades software se dividen en módulos independientes:

- **Planificador de horarios:** activa eventos de dispensación consultando el reloj del sistema (`time`, `schedule` o `APScheduler`).
- **Controlador del servo:** genera señal PWM y posiciona el carrusel semanal mediante `RPi.GPIO` o `pigpio`.
- **Gestor HX711:** adquiere lecturas de peso, aplica calibración y evalúa el umbral de detección.
- **Sistema de logs:** registra eventos con marca temporal UTC en formato JSON o CSV.
- **Gestor de tolerancia a fallos:** almacena mensajes pendientes cuando existe pérdida de red y los reenvía al restaurarse la conectividad.

4.5. Diagrama de Bloques de Hardware

La Figura 1 ilustra la interconexión lógica y física entre los módulos del sistema.

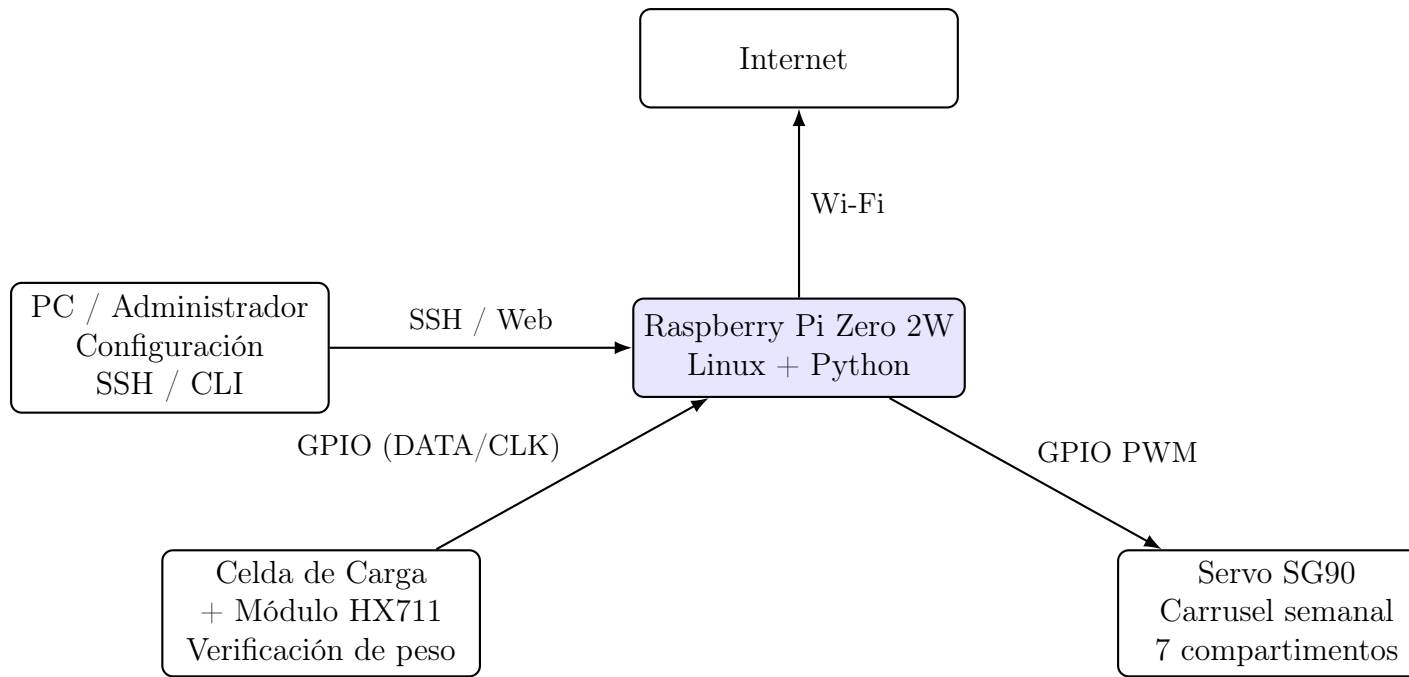


Figura 1: Diagrama de bloques de hardware del sistema propuesto.

4.6. Arquitectura Software del Sistema

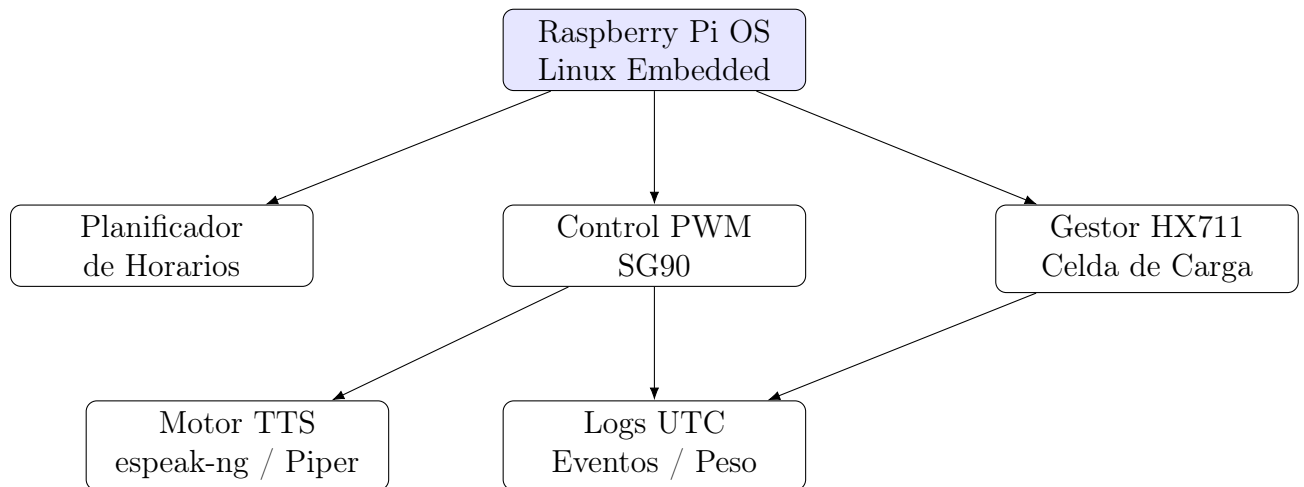


Figura 2: Arquitectura software propuesta.

4.7. Interfaces Eléctricas

La Raspberry Pi Zero 2W se conecta directamente con todos los periféricos mediante GPIO. La Tabla 2 resume una asignación sugerida de pines.

Tabla 2: Asignación sugerida de GPIO

GPIO	Elemento	Función
GPIO18	SG90	PWM del servomotor
GPIO5	HX711 DATA	Lectura de datos del ADC
GPIO6	HX711 CLK	Reloj del protocolo HX711
GPIO27	Pulsador	Confirmación manual / mantenimiento
GPIO21	Audio PWM	Salida auxiliar audio

Se recomienda utilizar la alimentación de 3.3V de la Raspberry Pi para el módulo HX711, dado que opera en ese rango lógico. El servomotor SG90 puede alimentarse directamente desde el pin 5V de la placa, aunque se recomienda una fuente dedicada si se prevén picos de corriente al iniciar el movimiento.

Todas las referencias de tierra (GND) deberán permanecer conectadas en común para garantizar estabilidad lógica.

5. Diagramas de Flujo del Sistema

5.1. Flujo Operativo Principal

La Figura 3 describe el ciclo de vida completo de un evento de dispensación, desde la espera del horario programado hasta el registro y notificación del resultado.

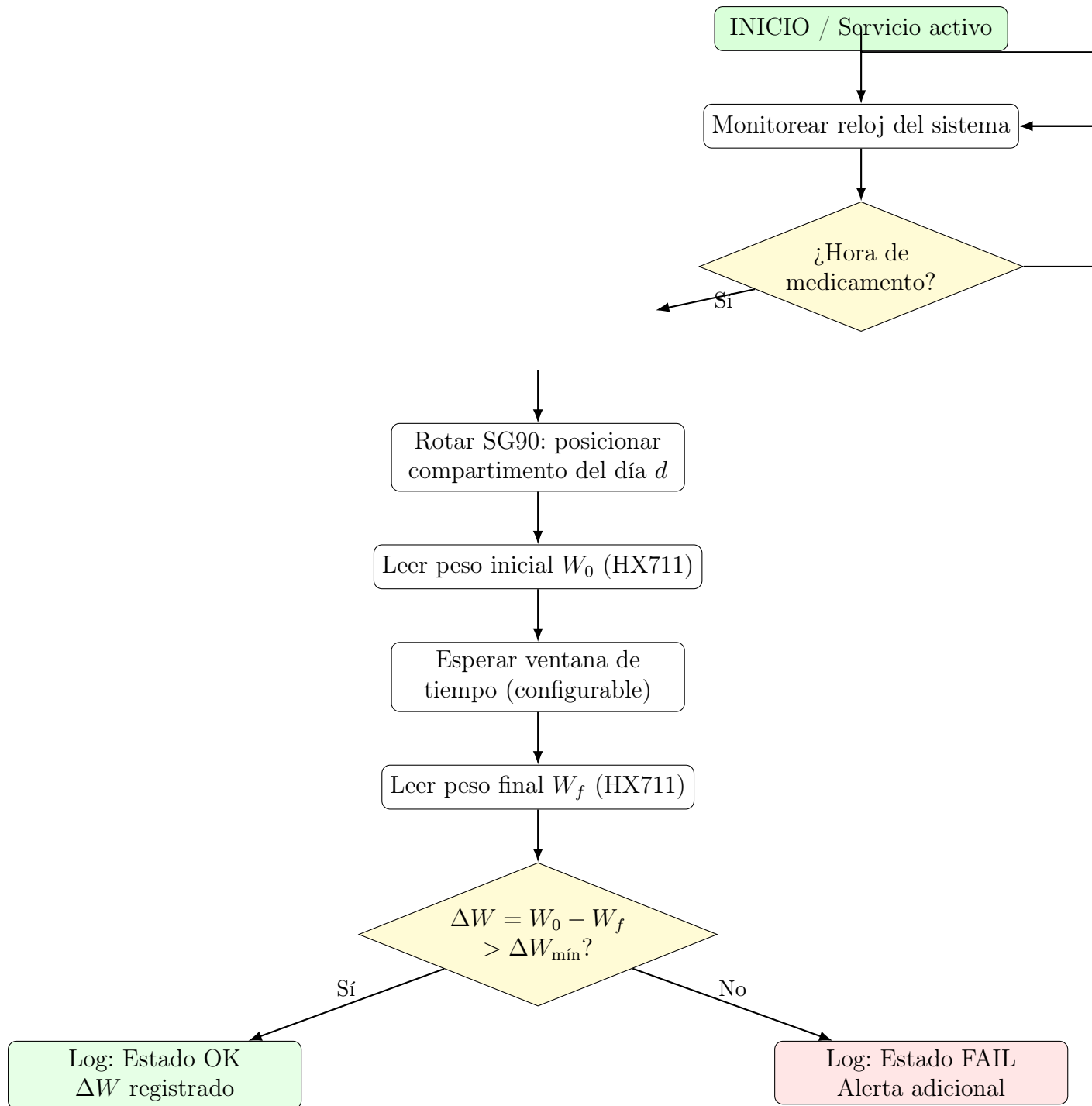


Figura 3: Diagrama de flujo operativo principal del sistema.

5.2. Flujo de Lectura y Calibración del HX711

La Figura 4 describe el proceso de inicialización, calibración y lectura de peso mediante el módulo HX711.

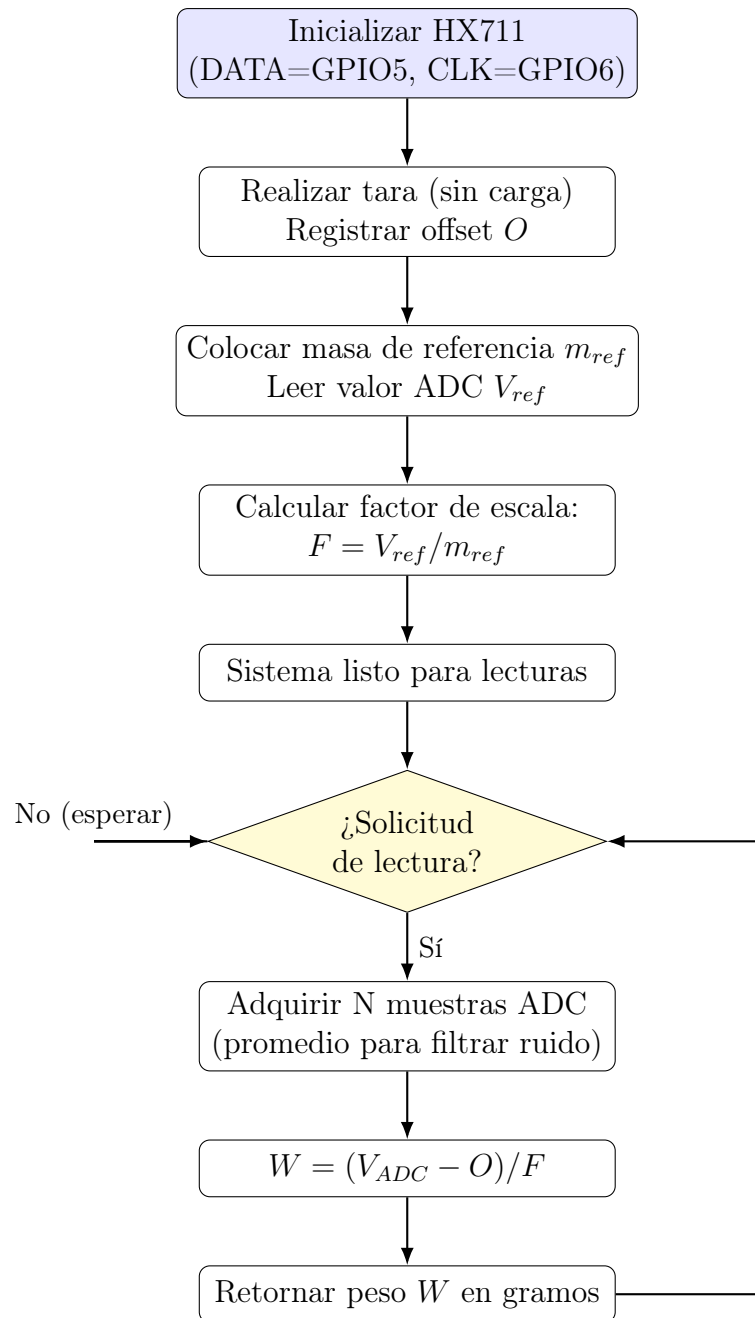


Figura 4: Diagrama de flujo de inicialización, calibración y lectura del módulo HX711.

5.3. Flujo de Notificación con Tolerancia a Fallos de Red

6. Plan de Verificación y Pruebas

Esta sección presenta un primer borrador del plan de verificación y pruebas unitarias del sistema. El objetivo es garantizar que cada módulo cumpla con los requerimientos funcionales y no funcionales especificados, tanto de forma aislada (pruebas unitarias) como integrada (pruebas de sistema).

6.1. Estrategia General de Pruebas

Las pruebas se organizan en tres niveles:

1. **Pruebas unitarias:** verificación del funcionamiento individual de cada módulo software en aislamiento, utilizando entradas controladas y simulando dependencias externas mediante *mocks* cuando sea necesario.
2. **Pruebas de integración:** verificación del comportamiento del sistema cuando dos o más módulos interactúan (por ejemplo, planificador + servo + HX711).
3. **Pruebas de sistema / aceptación:** validación del flujo completo de dispensación bajo condiciones reales, incluyendo conectividad y hardware físico.

Se utilizará el framework `pytest` como herramienta principal para las pruebas automatizadas.

6.2. Pruebas Unitarias por Módulo

6.2.1. PU-01: Módulo de Control del Servomotor SG90

Tabla 3: Pruebas unitarias – Módulo servo SG90

ID	Descripción	Entrada	Resultado esperado	RF/RNF
PU-01.1	Posicionamiento en compartimento día 0	Día = 0	Ángulo = 0°	RF-2
PU-01.2	Posicionamiento en compartimento día 3	Día = 3	Ángulo = 154.3°	RF-2
PU-01.3	Posicionamiento en compartimento día 6	Día = 6	Ángulo = 308.6°	RF-2
PU-01.4	Tiempo de respuesta del movimiento	Evento activado	Movimiento $\leq$ 200 ms	RNF-2
PU-01.5	Valor fuera de rango	Día = 7	Excepción <code>ValueError</code>	RF-2

6.2.2. PU-02: Módulo HX711 / Celda de Carga

Tabla 4: Pruebas unitarias – Módulo HX711

ID	Descripción	Entrada	Resultado esperado	RF/RNF
PU-02.1	Calibración con masa conocida	Masa ref. = 100 g	Factor de escala calculado correctamente	RF-8
PU-02.2	Lectura de peso en reposo	Compartimento vacío	$W \approx 0 \text{ g } (\pm 1 \text{ g})$	RNF-8
PU-02.3	Detección de medicamento	Masa = 2 g (pastilla)	$\Delta W > \Delta W_{\min} = 1 \text{ g}$	RF-3
PU-02.4	Estabilidad de lectura repetida	10 lecturas sucesivas	Desviación estándar $< 0,5 \text{ g}$	RNF-8
PU-02.5	Respuesta a tara	Tara ejecutada	Offset = 0 g	RF-8

6.2.3. PU-04: Módulo de Planificador de Horarios

Tabla 5: Pruebas unitarias – Planificador de horarios

ID	Descripción	Entrada	Resultado esperado	RF/RNF
PU-04.1	Activación de evento en horario	Hora simulada = horario	Evento disparado correctamente	RF-2
PU-04.2	Modificación dinámica de horario	Nuevo horario en caliente	Sin reinicio de servicio	RF-5
PU-04.3	Múltiples horarios en un día	3 horarios configurados	Los 3 eventos se activan	RF-2
PU-04.4	Persistencia de horarios	Reinicio del sistema	Horarios cargados desde archivo	RF-6

6.2.4. PU-06: Módulo de Registro de Eventos (Logs)

Tabla 6: Pruebas unitarias – Sistema de logs

ID	Descripción	Entrada	Resultado esperado	RF/RNF
PU-06.1	Registro de evento exitoso	Evento OK	Entrada en log con marca UTC	RF-6
PU-06.2	Registro de evento fallido	Evento FAIL	Entrada en log con marca UTC	RF-6
PU-06.3	Registro de peso	$W_0, W_f, \Delta W$	Valores incluidos en el log	RF-6
PU-06.4	Formato de marca temporal	Evento generado	Timestamp en formato ISO 8601 UTC	RF-6

6.3. Pruebas de Integración

Tabla 7: Pruebas de integración del sistema

ID	Descripción	Módulos involucrados	Resultado esperado
PI-01	Ciclo completo de dispensación	Scheduler + Servo + HX711 + TTS + Log	Flujo sin interrupciones, log y notificación generados
PI-02	Detección de no retiro	HX711 + Log	Estado FAIL registrado y notificado
PI-03	Tolerancia a pérdida de red	Cola	Mensajes encolados y re-enviados al restaurar
PI-04	Cambio de horario en caliente	Scheduler + CLI	Nuevo horario activo sin reinicio

6.4. Criterios de Aceptación

Para la aprobación del sistema se definen los siguientes criterios mínimos:

- El 100 % de las pruebas unitarias (PU) deben pasar sin errores.
- El 100 % de las pruebas de integración (PI) deben pasar sin errores.
- La latencia de movimiento del servo debe ser inferior a 200 ms (RNF-2) en al menos 95 de 100 ejecuciones.
- La detección de retiro mediante HX711 debe ser correcta en al menos 95 % de los eventos de prueba (con masas de referencia de 1 g, 2 g y 5 g).
- El sistema debe recuperarse automáticamente ante reinicios forzados del servicio (`systemd`) en menos de 10 segundos (RNF-7).